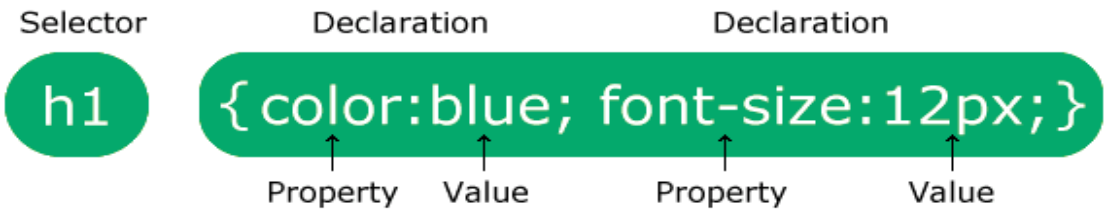
CSS

CSS is the language we use to style a Web page.

- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media
- CSS saves a lot of work. It can control the layout of multiple web pages all at once
- External stylesheets are stored in CSS files
- It is a language that is used to describe the style of a document.
- Current version is CSS3

Css syntax :



External CSS

With an external style sheet, you can change the look of an entire website by changing just one file!

Each HTML page must include a reference to the external style sheet file inside the <link> element, inside the head section.

Internal CSS

An internal style sheet may be used if one single HTML page has a unique style.

The internal style is defined inside the <style> element, inside the head section.

Inline CSS

An inline style may be used to apply a unique style for a single element.

To use inline styles, add the style attribute to the relevant element. The style attribute can contain any CSS property.

CSS selectors :

CSS selectors are used to "find" (or select) the HTML elements you want to style.

- CSS element Selector

  The element selector selects HTML elements based on the element name.

- CSS id Selector

- The id selector uses the id attribute of an HTML element to select a specific element.

- The id of an element is unique within a page, so the id selector is used to select one unique element!

- To select an element with a specific id, write a hash (#) character, followed by the id of the element.

- CSS class Selector

- The class selector selects HTML elements with a specific class attribute.

- To select elements with a specific class, write a period (.) character, followed by the class name.

- CSS Universal Selector

  The universal selector (*) selects all HTML elements on the page.

- The CSS Grouping Selector

- The grouping selector selects all the HTML elements with the same style definitions.

- It will be better to group the selectors, to minimize the code.

- To group selectors, separate each selector with a comma.

CSS Combinators

A combinator is something that defines the relationship between two or more selectors.

- Descendant combinator (space)
- Child combinator (>)
- Next sibling combinator (+)
- Subsequent-sibling combinator (~)

Descendant Combinator (space)

The descendant combinator matches all elements that are descendants (children, grandchildren, etc.) of a specified element.

Child Combinator (>)

The child combinator selects all elements that are direct children of a specified element.

Next Sibling Combinator (+)

- The next sibling combinator is used to select an element that is directly after a specific element.
- Sibling elements must have the same parent element.

Subsequent-sibling Combinator (~)

The subsequent-sibling combinator selects all elements that are next siblings of a specified element.

CSS !important Rule

The `!important` rule is used to give the value of a specific property the highest priority.

The `!important` rule will override ALL previous styling rules for that specific property on that element!

The `!important` keyword is added to the end of a CSS declaration, before the semicolon.

Syntax :

```
selector {
  property: value !important;
}
```

Comments

- Comments are used to explain the CSS code, and may help when you edit the source code at a later date.
- Comments are also used to temporarily disable sections of CSS code within a stylesheet.
- Comments are ignored by browsers!
- A CSS comment is placed inside the HTML <style> element, and starts with /* and ends with */:

Text Properties

The `text-align` property is used to set the horizontal alignment of a text.

```
text-align: left|right|center|justify|initial|inherit;
```

The **text-align-last** property specifies how to align the last line of a text.

```
text-align-last: auto|left|right|center|justify|start|end|initial|inherit;
```

The **direction** property specifies the text direction/writing direction within a block-level element.

```
direction: ltr|rtl|initial|inherit;
```

The CSS `text-decoration` property is used to control the appearance of decorative lines on text.

It is a shorthand property for the following individual properties:

- `text-decoration-line`
- `text-decoration-color`
- `text-decoration-style`
- `text-decoration-thickness`

The CSS `text-decoration-line` property sets the type of decoration line added to the text.

`text-decoration-line: none|underline|overline|line-through|initial|inherit;`

The CSS `text-decoration-color` property is used to set the color of the decoration line.

`text-decoration-color: color|initial|inherit;`

The **`text-decoration-style`** property sets the style of the text decoration (like solid, wavy, dotted, dashed, double).

`text-decoration-style: solid|double|dotted|dashed|wavy|initial|inherit;`

The **`text-decoration-thickness`** property specifies the thickness of the decoration line.

`text-decoration-thickness: auto|from-font|`*`length/percentage`*`|initial|inherit;`

The CSS **`text-transform`** property is used to control the capitalization of text in an element.

It can be used to transform a text into uppercase or lowercase letters, or capitalize the first letter of each word, without changing the original content in HTML.

`text-transform: none|capitalize|uppercase|lowercase|initial|inherit;`

The CSS **`text-indent`** property is used to specify the indentation of the first line in a text-block.

`text-indent: length|initial|inherit;`

The CSS **`letter-spacing`** property is used to specify the space between the characters in a text.

`letter-spacing: normal|length|initial|inherit;`

The CSS **`line-height`** property is used to specify the space between lines.

`line-height: normal|number|length|initial|inherit;`

The CSS **`word-spacing`** property is used to specify the space between the words in a text.

```
word-spacing: normal|length|initial|inherit;
```

The `text-shadow` property adds shadow to text.

This property accepts a comma-separated list of shadows to be applied to the text.

```
text-shadow: h-shadow v-shadow blur-radius color|none|initial|inherit;
```

Font Properties

The `font-family` property specifies the font for an element.

```
font-family: family-name|generic-family|initial|inherit;
```

The CSS **`font-style`** property specifies the font style for a text..

```
font-style: normal|italic|oblique|initial|inherit;
```

The CSS **`font-weight`** property specifies how thick or thin characters in text should be displayed.

```
font-weight: normal|bold|bolder|lighter|number|initial|inherit;
```

The CSS `font-size` property is used to specify the size of the text/font.

```
font-size:medium|xx-small|x-small|small|large|x-large|xx-
large|smaller|larger|length|initial|inherit;
```

The CSS overflow Property

The CSS overflow property controls what happens to content that is too big to fit into an area.

It specifies whether to clip the content or to add scrollbars when the content of an element is too big.

The overflow property has the following values:

- visible - Default. The overflow is not clipped. The content renders outside the element's box
- hidden - The overflow is clipped, and the rest of the content is hidden
- scroll - Scrollbars are added. User must scroll to see all content
- auto - Similar to scroll, but adds scrollbars only when necessary

The `overflow-x` and `overflow-y` properties specifies whether to change the overflow of content just horizontally or vertically (or both):

- `overflow-x` specifies what to do with the left/right edges of the content.
- `overflow-y` specifies what to do with the top/bottom edges of the content.

CSS Colors

In CSS, colors are specified by using a predefined color name, or with a RGB, HEX, HSL, RGBA, HSLA value.

- **CSS Color Names**

In CSS, a color can be specified by using a predefined color name ex.., Red, blue, green, yellow, white

- **RGB Value**

An RGB color value represents RED, GREEN, and BLUE light sources.

rgb(*red, green, blue*)

Each parameter (red, green, and blue) defines the intensity of the color between 0 and 255.

For example, rgb(255, 0, 0) is displayed as red, because red is set to its highest value (255) and the others are set to 0.

To display black, set all color parameters to 0, like this: rgb(0, 0, 0).

To display white, set all color parameters to 255, like this: rgb(255, 255, 255).

- **RGBA Value**

RGBA color values are an extension of RGB color values with an alpha channel - which specifies the opacity for a color.

rgba(*red, green, blue, alpha*)

The alpha parameter is a number between 0 (fully transparent) and 1 (not transparent at all)

- **HEX Value**

A hexadecimal color is specified with: #RRGGBB.

#rrggbb

Where rr (red), gg (green) and bb (blue) are hexadecimal values between 00 and ff (same as decimal 0-255).

Sometimes you will see a 3-digit hex code in the CSS source.

The 3-digit hex code is a shorthand for some 6-digit hex codes.

The 3-digit hex code has the following form:

#rgb

Where r, g, and b represent the red, green, and blue components with values between 0 and f.

The 3-digit hex code can only be used when both the values (RR, GG, and BB) are the same for each component. So, if we have #ff00cc, it can be written like this: #f0c.

- **HSL Value**

HSL stands for hue, saturation, and lightness.

hsl(*hue, saturation, lightness*)

Hue is a degree on the color wheel (from 0 to 360):

- 0 (or 360) is red
- 120 is green
- 240 is blue

Saturation is a percentage value: 0% means a shade of gray, and 100% is the full color.

Lightness is also a percentage; 0% is black, 50% is neither light or dark, 100% is white.

Saturation

Saturation can be described as the intensity of a color.

100% is pure color, no shades of gray.

50% is 50% gray, but you can still see the color.

0% is completely gray; you can no longer see the color.

Lightness

The lightness of a color can be described as how much light you want to give the color, where 0% means no light (black), 50% means 50% light (neither dark nor light) and 100% means full lightness (white).

- **HSLA Value**

HSLA color values are an extension of HSL color values with an alpha channel - which specifies the opacity for a color.

hsla(*hue, saturation, lightness, alpha*)

The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (not transparent at all)

CSS Gradients

The CSS gradient functions let you display smooth transitions between two or more colors within an element.

CSS defines three types of gradients:

- **Linear Gradients** - The color transition goes down, up, left, right, or diagonally
- **Radial Gradients** - The color transition goes out from a central point
- **Conic Gradients** - The color transition is rotated around a center point

The CSS gradient functions are used within the background-image property.

➢ CSS linear-gradient() Function

The CSS linear-gradient() function creates a linear gradient.

A linear gradient defines a color transition that goes in a straight line, it can go down, up, to left, to right, or diagonally.

A linear gradient requires at least two color stops. Color stops are the colors you want to render smooth transitions among. You can also set a starting point and a direction (or an angle) along with the gradient effect.

background-image: linear-gradient(*direction, color-stop1, color-stop2, ...*);

Linear Gradient - Using Angles

If you want more control over the direction of the gradient, you can define an angle parameter, instead of the predefined directions (to bottom, to top, to right, to left, to bottom right, etc.).

- A value of 0deg is equivalent to "to top".
- A value of 90deg is equivalent to "to right".
- A value of 180deg is equivalent to "to bottom".
- A value of 270deg equivalent to "to left"

The CSS repeating-linear-gradient() function is used to repeat linear gradients:

➢ CSS radial-gradient() Function

The CSS radial-gradient() function creates a radial gradient.

A radial gradient defines a color transition that goes out from a central point.

background-image: radial-gradient(*shape size* at *position, start-color, ..., last-color*);

By default, shape is ellipse, size is farthest-corner, and position is center.

The shape parameter defines the shape of the gradient. It can take one of the following values:

- **ellipse** (this is default)
- **circle**

The CSS repeating-radial-gradient() function is used to repeat radial gradients

➢ CSS conic-gradient() Function

The CSS conic-gradient() function creates a conic gradient.

A conic gradient is a gradient with color transitions rotated around a center point.

background-image: conic-gradient([from *angle*] [at *position*,] *color* [*degree*], *color* [*degree*], ...);

The CSS repeating-conic-gradient() function is used to repeat conic gradients

Units

CSS Units are used to specify the size of elements, spacing, and layout measurements in CSS.

CSS units are used to define the length and size of several properties.

Several CSS properties take "length" values, such as font-size, width, margin, padding, border, etc.

Absolute Units

Absolute units have **fixed values** and do not change depending on screen size.

Absolute units do NOT change when the screen size change, and are not recommended for websites. However, they can be used if the output medium is known, such as for print layout.

Unit	Meaning
px	pixels
cm	centimeters
mm	millimeters
in	inches
pt	points
pc	picas

CSS Relative Units

Relative units depend on **another element or screen size**. (like parent element, root element, or viewport).

Relative length units scale better between different screen sizes, making them ideal for responsive web design.

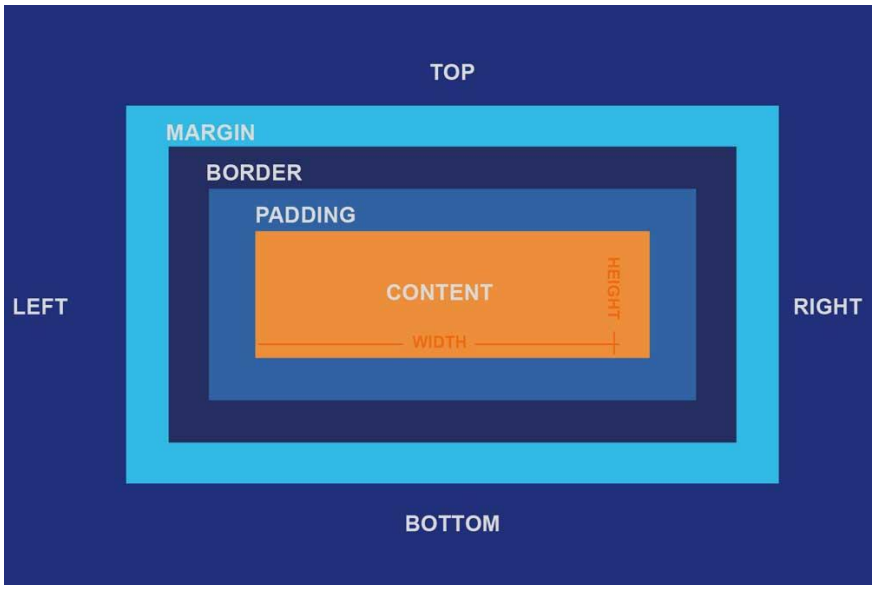
Unit	Relative To
%	parent element
em	parent font size
rem	root font size
vw	viewport width
vh	viewport height

Box Model

The CSS box model is essentially a box that wraps around every HTML element.

Every box consists of four parts: content, padding, borders and margins.

- Content is the **actual data inside the element**.
- Padding is the **space between content and border**, It creates **inner space**.
- Border surrounds the **padding and content**.
- Margin is the **space outside the border**, It separates elements from each other.



When you set the width and height properties of an element with CSS, you just set the width and height of the **content area**. To calculate the total width and height of an element, you must also include the padding and borders.

The total width of an element should be calculated like this:

Total element width = width + left padding + right padding + left border + right border

The total height of an element should be calculated like this:

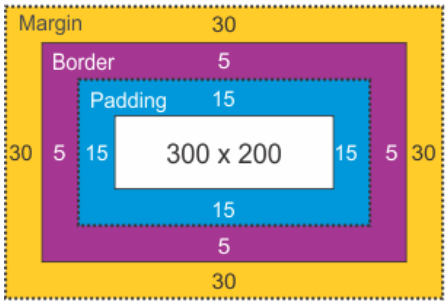
Total element height = height + top padding + bottom padding + top border + bottom border

The CSS box-sizing property defines how to calculate the width and height of an element: should the calculation include padding and borders, or not.

If you set box-sizing: border-box; on an element, the padding and border are included in the calculation of the width and height:

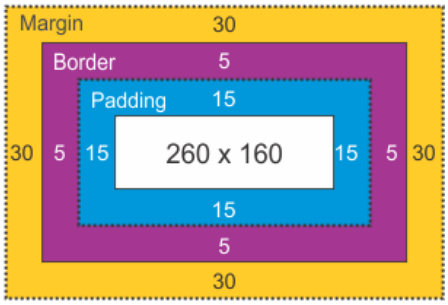
box-sizing: content-box|border-box|initial|inherit;

Box Model is `content-box`




```
div{
      width: 300px;
      height: 200px;
      padding: 15px;
      border: 5px solid grey;
      margin: 30px;
      -moz-box-sizing: content-box;
      -webkit-box-sizing: content-box;
      box-sizing: content-box;
    }
```

Box Model is `border-box`




```
div{
      width: 300px;
      height: 200px;
      padding: 15px;
      border: 5px solid grey;
      margin: 30px;
      -moz-box-sizing: border-box;
      -webkit-box-sizing: border-box;
      box-sizing: border-box;
    }
```

CSS Padding

The CSS padding properties are used to generate space around an element's content, inside of any defined borders.

CSS has properties for specifying the padding for each side of an element:

- padding-top - sets the top padding of an element
- padding-right - sets the right padding of an element
- padding-bottom - sets the bottom padding of an element
- padding-left - sets the left padding of an element

The padding property is a shorthand property for the following individual padding properties:

- padding-top
- padding-right
- padding-bottom
- padding-left

padding: *length*|initial|inherit;

(TRBL, T(R/L)B, (T/B)(L/R))

CSS Borders

The CSS border properties allow you to specify the style, width, and color of an element's border.

The border-style property sets the style of an element's four borders. This property can have from one to four values.

border-style: none|hidden|dotted|dashed|solid|double|groove|ridge|inset|outset|initial|inherit;

The border-width property sets the width of an element's four borders. This property can have from one to four values.

border-width: medium|thin|thick|*length*|initial|inherit;

The border-color property sets the color of an element's four borders. This property can have from one to four values.

border-color: *color*|transparent|initial|inherit;

The border property is a shorthand property for:

- border-width
- border-style (required)
- border-color

border: *border-width border-style border-color*|initial|inherit;

The border-radius property is used to add rounded borders to an element

- **Four Values (Individual):** border-radius: 10px 20px 30px 40px;
    - *Order:* Top-left, top-right, bottom-right, bottom-left.
- **Three Values:** border-radius: 10px 20px 30px;

    - *Order:* Top-left, Top-right & Bottom-left, Bottom-right.

- **Two Values:** border-radius: 10px 20px;
    - *Order:* Top-left & Bottom-right, Top-right & Bottom-left.

2. Individual Longhand Properties

For maximum control or readability, use specific properties to set each corner separately:

- border-top-left-radius: 10px;
- border-top-right-radius: 20px;
- border-bottom-right-radius: 30px;
- border-bottom-left-radius: 40px;

The CSS margin properties are used to create space around elements, outside of any defined borders.

- margin-top - sets the top margin of an element
- margin-right - sets the right margin of an element
- margin-bottom - sets the bottom margin of an element
- margin-left - sets the left margin of an element

The margin property is a shorthand property for the following individual margin properties-+

Margin collapse is when two margins collapse into a single margin.

Top and bottom margins of elements are sometimes collapsed into a single margin that is equal to the largest of the two margins.

CSS Backgrounds

The background of an element is the total size of the element, including padding and border (but not the margin).

The CSS background properties are used to add background effects for elements.

The **background-color** property sets the background color of an element.

The **background-image** property specifies an image to use as the background of an element.

The **background-repeat** property sets if/how a background image will be repeated.

background-repeat: repeat|repeat-x|repeat-y|no-repeat|initial|inherit;

The background-position property is used to set the starting position of the background image.(top, bottom, left, right, center)

The background-attachment property specifies whether the background image should scroll or be fixed (will not scroll with the rest of the page):

background-attachment: scroll|fixed|local|initial|inherit;

The **Background Size** property sets custom sizes for the background image.

background-size: cover ensures the entire container is filled, even if parts of the image are cropped, while background-size: contain ensures the entire image is visible, which may leave empty space in the container

(cover – fill entire area, contain – fit inside)

background: [background-color] [background-attachment] [background-image] [background-repeat] [background-position];

CSS Positioning

CSS positioning is about controlling the placement of elements within a web page.

CSS positioning is used to control the position of elements on a webpage. (top, bottom, left, right)

The `position` property specifies the positioning type for an element.

CSS position: static

All HTML elements are positioned static by default.

Static positioned elements are not affected by the top, bottom, left, and right properties.

An element with `position: static;` is always positioned according to the normal flow of the page

CSS position: relative

An element with `position: relative;` is positioned relative to its normal position in the document flow.

Setting the top, right, bottom, and left properties will cause the element to be adjusted away from its normal position. Other content will not be adjusted to fit into any gap left by the element.

CSS position: fixed

An element with `position: fixed;` is positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled. The top, right, bottom, and left properties are used set the final location of the element.

A fixed element does not leave a gap in the page where it would normally have been located.

CSS position: absolute

An element with `position: absolute;` is positioned relative to the nearest positioned ancestor (with position other than static).

However; if an absolute positioned element has no positioned ancestors, it uses the document body, and moves along with page scrolling.

Note: Absolute positioned elements are removed from the normal document flow, and can overlap other elements.

CSS position: sticky

An element with `position: sticky;` toggles between a `relative` and `fixed` position, depending on the scroll position.

A sticky element is positioned relative until a certain scroll position is reached - then it "sticks" in that place (like position:fixed).

z-index controls the vertical stacking order of elements (which element appears on top).

Higher z-index → comes forward

Lower z-index → goes backward

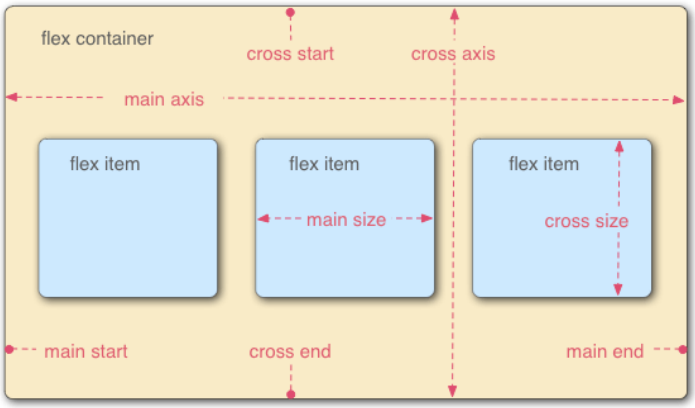
z-index works only when:

```
position: relative / absolute / fixed / sticky
```

Flexbox : is a one-dimensional layout system used to arrange items in rows or columns.

CSS Flexbox is short for the CSS Flexible Box Layout module.

Flexbox is a layout model for arranging items (horizontally or vertically) within a container, in a flexible and responsive way.



CSS Flex Container Properties

- `display` - Must be set to `flex` or `inline-flex`

The `flex-direction` property specifies the display-direction of flex items in the flex container.

This property can have one of the following values:

- `row` (default)
- `column`
- `row-reverse`
- `column-reverse`

The `flex-wrap` property specifies whether the flex items should wrap or not, if there is not enough room for them on one flex line.

This property can have one of the following values:

- `nowrap` (default)
- `wrap`
- `wrap-reverse`

The `flex-flow` property is a shorthand property for setting both the `flex-direction` and `flex-wrap` properties.

The **`justify-content`** property aligns the flex items along the main-axis (horizontally).

This property can have one of the following values:

- `center`
- `flex-start` (default)
- `flex-end`
- `space-around`
- `space-between`
- `space-evenly`

The **`align-items`** property aligns the flex items vertically (on the cross-axis).

This property can have one of the following values:

- `normal` (default)
- `stretch`
- `center`
- `flex-start`
- `flex-end`
- `baseline`

The **`align-content`** property aligns the flex lines. It only has effect when flex items wrap onto multiple lines.

This property can have one of the following values:

- `stretch` (default)
- `center`
- `flex-start`
- `flex-end`
- `space-between`
- `space-around`
- `space-evenly`

The `order` property specifies the display order of the flex items inside the flex container.

The first flex item in the source code does not have to appear as the first item in the layout.

The order value must be a number, and the default value is 0.

The `flex-grow` property specifies how much a flex item will grow relative to the rest of the flex items.

The value must be a number, and the default value is 0.

The `flex-shrink` property specifies how much a flex item will shrink relative to the rest of the flex items.

The value must be a number, and the default value is 1.

The `flex-basis` property specifies the initial length of a flex item.

When flex-direction is row (default), flex-basis affects width. When flex-direction is column, it affects height.

The `flex` property is a shorthand property for the `flex-grow`, `flex-shrink`, and `flex-basis` properties.

The **`align-self`** property specifies the alignment for the selected item inside the flexible container.

This property overrides the default alignment set by the container's `align-items` property.

CSS Media Queries

CSS media queries allow you to apply styles based on the characteristics of a device or the environment displaying the web page.

CSS media queries are essential for creating responsive web pages.

The CSS @media rule is used to add media queries to your style sheet.

Media Queries in CSS allow us to change styles depending on screen size. They are the foundation of responsive web design, which ensures your website looks good on desktops, tablets, and mobiles without building separate versions.

```
@media media-type and (condition) {

  /* CSS rules */

}
```

- @media → Starts the query.
- media-type → The type of output device (commonly screen for devices, print for printing).
- condition → The rule to check (like max-width: 800px). If true → styles inside apply.

not: This optional keyword inverts the meaning of the entire media query.

and: This keyword combines a media-type and one or more media-features.

CSS Media Types

The optional media type specifies the type of media the styles are for. If media type is omitted, it will be set to "all".

Value	Description
all	Used for all media type devices
print	Used for print preview mode
screen	Used for computer screens, tablets, and smart-phones

CSS Media Features

The media feature specifies a specific characteristic of the device.

Value	Description
max-height	Maximum height of the viewport
min-height	Minimum height of the viewport
height	Height of the viewport (including scrollbar)
max-width	Maximum width of the viewport
min-width	Minimum width of the viewport
width	Width of the viewport (including scrollbar)
orientation	Orientation of the viewport (landscape or portrait)

CSS Grid Layout Module

The Grid Layout Module offers a grid-based layout system, with rows and columns.

The Grid Layout Module allows developers to easily create complex web layouts.

Grid vs. Flexbox

CSS Grid is used for two-dimensional layout, with rows AND columns.

CSS Flexbox is used for one-dimensional layout, with rows OR columns.

CSS Grid Components

A grid always consists of:

- **A Grid Container** - The parent (container) element, where the `display` property is set to `grid` or `inline-grid`
- **One or more Grid Items** - The direct children of the grid container automatically becomes grid items

The CSS grid-template-columns Property

The `grid-template-columns` property defines the number and width of the columns in the grid.

The value is a space-separated-list, where each value defines the width of the respective column.

Common values are:

- Fixed lengths (100px 300px 200px)
- Percentages (20% 60% 20%)
- fr unit (1fr 2fr 1fr)
- auto (auto auto auto)
- repeat() (repeat(3, 1fr))

The CSS grid-template-rows Property

The `grid-template-rows` property defines the number and height of the rows in the grid.

The value is a space-separated-list, where each value defines the height of the respective row.

CSS Grid Gaps

The space between the rows and columns in a grid container are called *gaps* .

The gaps are created between the grid rows and columns, not on the outer edges of the grid container.

The `column-gap` property specifies the gap between the columns in a grid.

The `row-gap` property specifies the gap between the rows in a grid.

The CSS gap Property

The `gap` property is a shorthand property for `row-gap` and `column-gap`.

If a single value is provided, it applies the same gap to both rows and columns.

If two values are provided, the first value sets the `row-gap`, and the second value sets the `column-gap`.

The CSS justify-content Property

The `justify-content` property is used to align the grid content when it does not use all available space on the main-axis (horizontally).

This property can have one of the following values:

- `space-evenly`
- `space-around`
- `space-between`
- `center`
- `start`
- `end`

The CSS align-content Property

The `align-content` property is used to align the grid content when it does not use all available space on the cross-axis (vertically).

This property can have one of the following values:

- `space-evenly`
- `space-around`
- `space-between`
- `center`
- `start`
- `end`

The `place-content` property is a shorthand property for the `align-content` and the `justify-content` properties.

If `place-content` has two values:

- `place-content: start center;` - the `align-content` value is 'start' and `justify-content` value is 'center'

If `place-content` has one value:

- `place-content: end;` - both `align-content` and `justify-content` values are 'end'

CSS grid-column-start and grid-column-end

The `grid-column-start` property specifies on which column-line the grid item will start.

The `grid-column-end` property specifies on which column-line the grid item will end.

The `grid-column` property is a shorthand property for the `grid-column-start` and the `grid-column-end` properties.

```
grid-column: grid-column-start / grid-column-end;
```

CSS grid-row-start and grid-row-end

The `grid-row-start` property specifies on which row-line the grid item will start.

The `grid-row-end` property specifies on which row-line the grid item will end.

The `grid-row` property is a shorthand property for the `grid-row-start` and the `grid-row-end` properties.

```
grid-row: grid-row-start / grid-row-end;
```

Naming Grid Items with grid-area

The CSS `grid-template-areas` is a grid container property, and it specifies areas within the grid layout.

You can name grid items by using the CSS `grid-area` property, and then reference to the name in the `grid-template-areas` property.

Each area is defined within apostrophes. The named grid items in each area is defined inside the apostrophes, separated by a space.

We can use a period sign (.) to refer to a grid item with no name:

The CSS justify-self Property

The `justify-self` property is used to align a grid item within its grid cell in the inline direction.

Specifies the horizontal alignment within a cell

This property can have one of the following values:

- `auto` (default)
- `normal`
- `stretch`
- `start`
- `left`
- `center`
- `end`
- `right`

The CSS align-self Property

The `align-self` property is used to align a grid item within its grid cell in the block direction.

Specifies the vertical alignment within a cell

This property can have one of the following values:

- `auto` (default)
- `normal`
- `stretch`
- `start`
- `end`
- `center`

`place-self` - Shorthand property for `align-self` and `justify-self`

If the place-self property has two values:

- place-self: start center;
  - align-self property value is 'start'
  - justify-self property value is 'center'

The CSS `order` property can be used to define the visual order of the grid items.

Transform

Transform is used to change the shape, size, or position of an element.

Function	Use
translate()	move element
rotate()	rotate
scale()	zoom in/out
skew()	tilt

Translate

Move element from its position - translate(x, y)

Transitions

The **CSS** transition **property** allows you to smoothly animate changes between styles over a given period of time.

Instead of instantly jumping from one style to another, transitions create a gradual effect, making web elements more interactive and visually appealing.

Syntax:

```
transition: property duration timing-function delay;
```

- **property** → the CSS property you want to animate (width, height, color, etc.). Use all to apply transitions to all animatable properties.
- **duration** → how long the transition takes (e.g., 2s, 500ms).
- **timing-function** → how the transition progresses over time (ease, linear, ease-in, ease-out, ease-in-out, or custom cubic-bezier).
- **delay** → wait before the transition starts.

The **timing-function** controls the speed curve of the transition:

- linear → same speed from start to end.
- ease → default, starts slow → speeds up → slows down.
- ease-in → slow at start.
- ease-out → slow at end.
- ease-in-out → slow at start and end.

Animations

CSS Animations help to animate elements from one frame to another. Controlling various CSS properties helps in changing the animation style and time without having the need to use JavaScript for the same.

Animations in CSS are built around two main components:

- @keyframes → Defines what changes happen during the animation.
- animation properties → Define how the animation runs (duration, timing, loop, etc.).

<u>example:</u>

```
@keyframes name {

  from { }

  to { }

}
```

Property	Use
animation-name	name
animation-duration	time
animation-iteration-count	repeat
animation-delay	delay
animation-direction	reverse

```
animation-direction: normal|reverse|alternate|alternate-reverse|initial|inherit;

animation-iteration-count: number|infinite|initial|inherit;

animation-timing-function: linear|ease|ease-in|ease-out|ease-in-out



animation: name duration timing-function delay iteration-count direction;
```

CSS Pseudo-classes

A CSS pseudo-class is a keyword that can be added to a selector, to define a style for a special state of an element.

In CSS, a pseudo-class is used to define the special state of an element.

Syntax

Pseudo-classes are always denoted by a single colon (:) followed by the pseudo-class name:

```
Selector : pseudo-class-name {
  CSS properties
}
```

Interactive Pseudo-classes

Interactive pseudo-classes apply styles based on **user interaction**:

- `:hover` - When mouse is over an element
- `:focus` - When an element has focus
- `:active` - When an element is being activated
- `:link` - Unvisited links
- `:visited` - Visited links

Structural Pseudo-classes

Structural pseudo-classes select elements based on **their position in the document tree**:

- `:first-child` - First child of a parent
- `:last-child` - Last child of a parent
- `:nth-child(n)` - The nth child of a parent
- `:lang()` - Elements with a specific language

Pseudo-elements

Pseudo-elements are used to style specific parts of an element or insert content using CSS.

```
Selector :: pseudo-element{
  property: value;
}
```

Pseudo-element	Use
`::before`	insert content before element
`::after`	insert content after element
`::first-letter`	style first letter
`::first-line`	style first line
`::selection`	style selected text
`::placeholder`	style input placeholder